ECE 526L Spring 2025 Lab 7

Design and Verification of a RAM and ROM



Muhammad Umar
CSUN ID: 204533244

Instructor: Prof. Orod Haghighiara

I hereby attest that this lab report is entirely my own work. I have not copied either code or text from anyone, nor have I allowed or will I allow anyone to copy my work.

Name (printed): **Muhammad Umar**

Name (signed):  Date: 04/10/2025

INTRODUCTION

This lab focused on the design and functional verification of register based Random Access Memory (**RAM**) and Read Only Memory (**ROM**) modules using Verilog. The primary objective is to develop two memory subsystems with parameterizable data and address widths, enabling flexibility in hardware designs.

The **RAM** module is designed to support both **read** and **write** operations, controlled through three signals: chip select (**cs**), output enable (**oe**), and write strobe (**ws**). A key feature of the RAM is its **bi-directional** data bus, implemented using an **inout** port. The design should behave that the module only drives the data bus during active read operations, while allowing external control for writes.

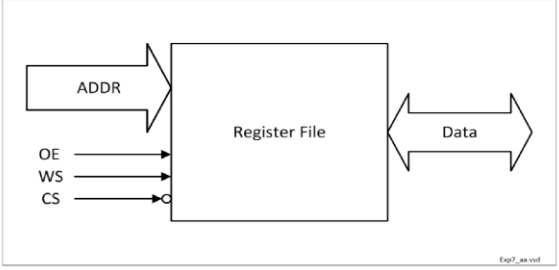


Figure 7.1 Block Diagram of RAM

The **ROM** module, on the other hand, is designed for read-only functionality and initialized using the Verilog system task **$readmemh**, which loaded memory contents from an external file named **rom_data.mem**. The ROM also included support for chip select(**cs**) and output enable(**oe**) signals, and correctly change its output to a high impedance (**z**) state when disabled.



Figure 7.2 Block Diagram of ROM

To verify the functionality of each design, dedicated testbenches needs to be developed. These testbenches should implement a variety of functional checks including:

a) Writing and reading all memory locations in RAM.
b) Ensuring proper high impedance behavior when oe or cs are inactive.
c) Validating a "**walking ones**" pattern to confirm bit level addressability.
d) Scrambling read data from ROM and storing it in a **mirror model** using a custom bit manipulation function.

The lab required not only the correct implementation of these modules but also careful validation through waveform analysis and logging which can be found in Appendices.

METHODOLOGY AND TEST PLAN

1. RAM Module (register_file_ram.v):

This module implements a **parameterizable register based RAM** which is capable of both read and write operations. It uses three primary control signals:

a) **cs (**chip select**)**:
It is active low and enables or disables access to the memory.

b) **oe (**output enable**)**:
It is active high and controls when data should be driven onto the bus.

c) **ws (**write strobe**)**:
It is triggered on the rising edge, and triggers a memory write operation.

The RAM has a **bi-directional** data port **(inout)**, allowing the same bus to be used for both input (during write) and output (during read). During a write operation, external logic drives the data bus, while during a read, the module itself drives the bus. When neither read nor write is active, the data lines are placed in a **high-impedance (z)** state as described in lab specification.
The use of always **@(posedge ws)** ensures synchronized memory writes, while a combinational always **@(*)** block handles asynchronous reads.

2. ROM Module (register_file_rom.v):

This module is initialized at simulation time using the **$readmemh** system task. The ROM reads data from a .mem formatted file named **rom_data.mem** and stores it in an internal memory array.
ROM control signals include:

a) **cs** (chip select, active low):
This Enables memory access.

b) **oe** (output enable, active high):
This Enables data output.

Unlike the RAM module, this ROM only supports **unidirectional output** through a standard output port. When not enabled, the output is driven to a **high-impedance state**, ensuring safe behavior in bus-based systems. The module is entirely combinational and does not include any edge-sensitive constructs since writes are not supported.

3. RAM Testbench:

This testbench verifies the functionality of the RAM module by simulating a required scenarios:

a) **Sequential Write:** Writes values 0 through 31 to all addresses in memory.

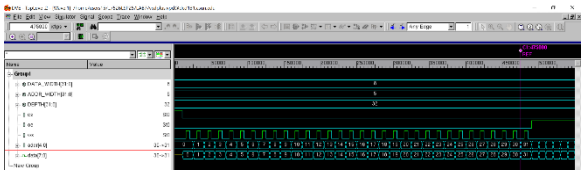


Figure 7.3 Waveform of Writing to RAM

```
Writing 0 to 31 into addresses 0 to 31 (sequential)
  Time 25000 | WROTE 0 at Address 0
  Time 40000 | WROTE 1 at Address 1
  Time 55000 | WROTE 2 at Address 2
  Time 70000 | WROTE 3 at Address 3
  Time 85000 | WROTE 4 at Address 4
  Time 100000 | WROTE 5 at Address 5
  Time 115000 | WROTE 6 at Address 6
  Time 130000 | WROTE 7 at Address 7
  Time 145000 | WROTE 8 at Address 8
  Time 160000 | WROTE 9 at Address 9
  Time 175000 | WROTE 10 at Address 10
  Time 190000 | WROTE 11 at Address 11
  Time 205000 | WROTE 12 at Address 12
  Time 220000 | WROTE 13 at Address 13
  Time 235000 | WROTE 14 at Address 14
  Time 250000 | WROTE 15 at Address 15
  Time 265000 | WROTE 16 at Address 16
  Time 280000 | WROTE 17 at Address 17
  Time 295000 | WROTE 18 at Address 18
  Time 310000 | WROTE 19 at Address 19
  Time 325000 | WROTE 20 at Address 20
  Time 340000 | WROTE 21 at Address 21
  Time 355000 | WROTE 22 at Address 22
  Time 370000 | WROTE 23 at Address 23
  Time 385000 | WROTE 24 at Address 24
  Time 400000 | WROTE 25 at Address 25
  Time 415000 | WROTE 26 at Address 26
  Time 430000 | WROTE 27 at Address 27
  Time 445000 | WROTE 28 at Address 28
  Time 460000 | WROTE 29 at Address 29
  Time 475000 | WROTE 30 at Address 30
  Time 490000 | WROTE 31 at Address 31
```

Figure 7.4 Log of Writing to RAM

b) **Read After Write:** Confirms that written data is accurately stored and can be retrieved without corruption.

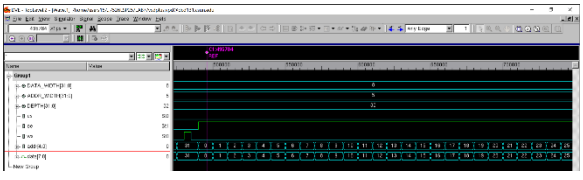


Figure 7.5 Waveform of Reading from RAM

```
 Time 490000 | WROTE 31 at Address 31

============================================================
Reading back values (verifying uniqueness for 32 locations)
============================================================
 Time 500000 | READ 0 from Address 0
 Time 510000 | READ 1 from Address 1
 Time 520000 | READ 2 from Address 2
 Time 530000 | READ 3 from Address 3
 Time 540000 | READ 4 from Address 4
 Time 550000 | READ 5 from Address 5
 Time 560000 | READ 6 from Address 6
 Time 570000 | READ 7 from Address 7
 Time 580000 | READ 8 from Address 8
 Time 590000 | READ 9 from Address 9
 Time 600000 | READ 10 from Address 10
 Time 610000 | READ 11 from Address 11
 Time 620000 | READ 12 from Address 12
 Time 630000 | READ 13 from Address 13
 Time 640000 | READ 14 from Address 14
 Time 650000 | READ 15 from Address 15
 Time 660000 | READ 16 from Address 16
 Time 670000 | READ 17 from Address 17
 Time 680000 | READ 18 from Address 18
 Time 690000 | READ 19 from Address 19
 Time 700000 | READ 20 from Address 20
 Time 710000 | READ 21 from Address 21
 Time 720000 | READ 22 from Address 22
 Time 730000 | READ 23 from Address 23
 Time 740000 | READ 24 from Address 24
 Time 750000 | READ 25 from Address 25
 Time 760000 | READ 26 from Address 26
 Time 770000 | READ 27 from Address 27
 Time 780000 | READ 28 from Address 28
 Time 790000 | READ 29 from Address 29
 Time 800000 | READ 30 from Address 30
 Time 810000 | READ 31 from Address 31

------------------------------------
```

Figure 7.6 Log of Reading from RAM

c) **Individual and Block Read:** Reads back each location separately and performs block reads with **oe** held high.

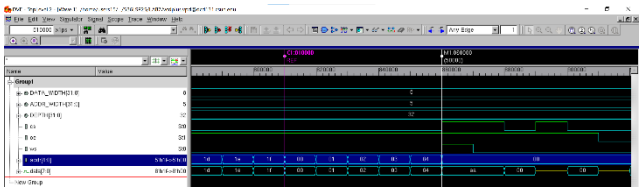


Figure 7.7 Waveform of Block Reading

```
==========================================
Block Read test (keep oe high)
==========================================
 Time 820000 | Output Enable(oe)=1 BLOCK READ Address 0 => 00
 Time 830000 | Output Enable(oe)=1 BLOCK READ Address 1 => 01
 Time 840000 | Output Enable(oe)=1 BLOCK READ Address 2 => 02
 Time 850000 | Output Enable(oe)=1 BLOCK READ Address 3 => 03
 Time 860000 | Output Enable(oe)=1 BLOCK READ Address 4 => 04

============================================================
```

Figure 7.8 Log of Block Reading

d) **Disabled State Check:** Attempts a write operation while **cs** is high to ensure memory remains unchanged.

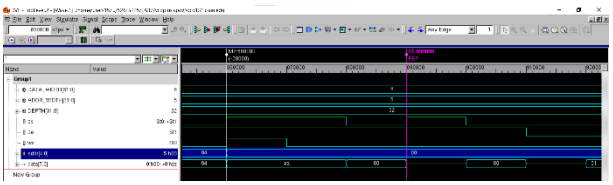


Figure 7.9 Waveform of Block Reading

```
============================================================
Testing disabled states (should ignore oe/ws when cs = 1)
============================================================
 Time 880000 | cs=1 writing = aa to Address = 0 should be ignored

------------------------------------------
Read back and confirm not written
------------------------------------------
 Time 890000 | Value at Address 0 (should still be 0): Data = 00
```

Figure 7.10 Log of Writing & Reading when cs = 1

e) **High-Impedance Condition:** Verifies that the data bus correctly transitions to High **z** when **cs** is deasserted or **oe** is low.

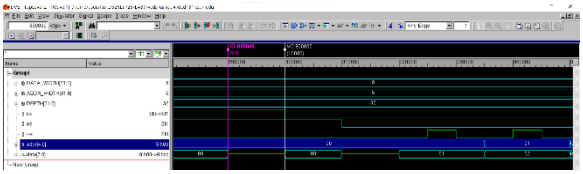


Figure 7.11 Waveform of high z checking

```
------------------------------------------
High-Z test When Chip Select is Active (cs=1)
------------------------------------------
 Time 900000 Chip Select(cs) = 1 | PASS: Data bus is high-Z as expected: zzzzzzzz

------------------------------------------------------------
Checking that data bus becomes High-Z when oe goes low (cs = 0)
------------------------------------------------------------
 Time 910000 | oe=1 Data driven = 00000000
 Time 920000 | oe=0 PASS: Data bus returned to High-Z: zzzzzzzz
```

Figure 7.12 Log of high z checking

f) **Walking Ones Test:** Writes a "walking ones" pattern (00000001, 00000010, ..., 10000000) to validate bit-level accuracy and address uniqueness.

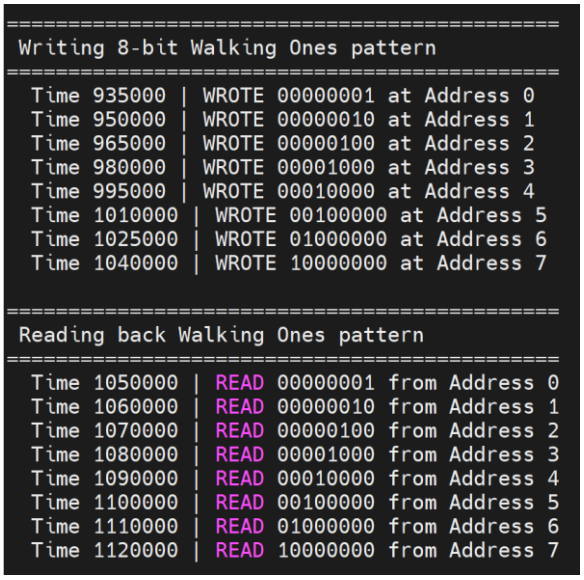


Figure 7.13 Log of reading and writing walking one test

4. ROM Testbench:

The ROM testbench focuses on verifying proper data initialization and controlled read behavior:

a) **Initialization Test:** Reads and prints contents from specific ROM addresses (0x04 to 0x0F) and other addresses till 0x1E. that were initialized via **rom_data.mem** using system task $**readmemh**.

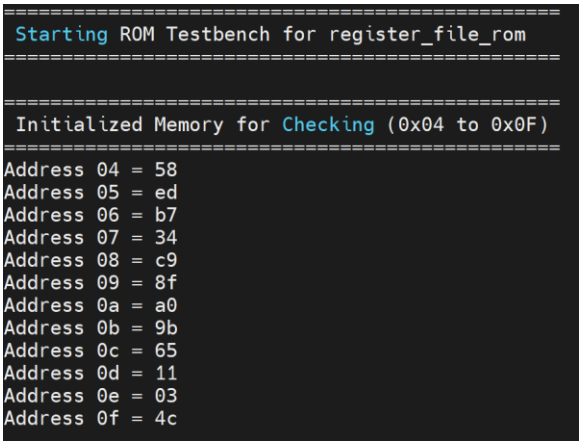


Figure 7.14 Log of ROM Values Checking

b) **Undefined Address Handling:** Attempts to read from locations (0x00 to 0x03) that are not initialized, expecting undefined (**x**) values.

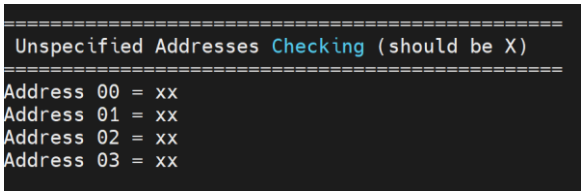


Figure 7.15 Log of Undefined Values Checking

c) **Data Scrambling:** Implements a **scramble function** that rearranges input bits from the ROM. For each byte in addresses 0x10 to 0x17, the bits are reordered from [7:0] to [0,7,1,6,2,5,3,4]. For example, a value like 8'hDA becomes 8'h73.

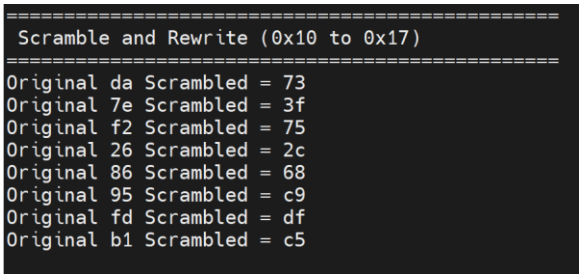


Figure 7.16 Log of Scrambled Values

d) **Full ROM Dump:** Iterates through the entire memory space to display the final state of the ROM contents.

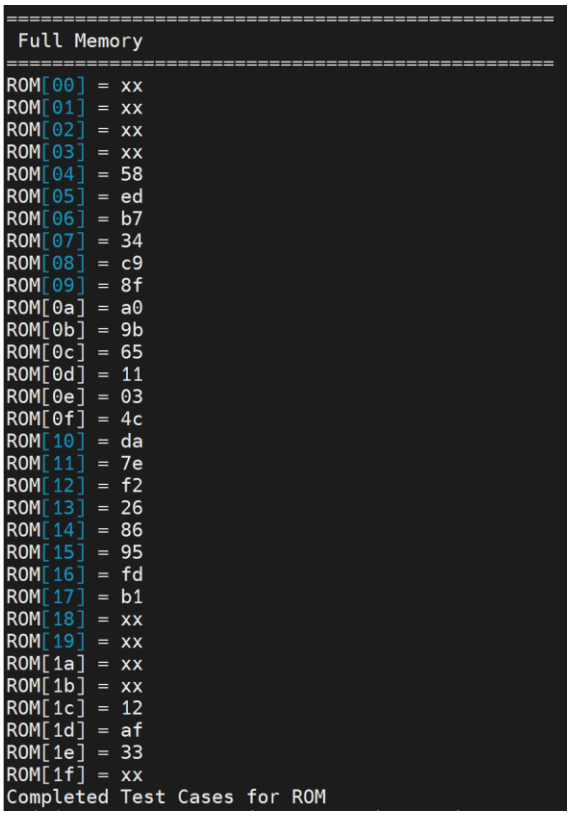


Figure 7.17 Log of Full ROM

This testbench thoroughly checks all specified ROM behaviors and includes visual logging and wave generation as well which is attached with lab files.

EXTRA CREDIT:

How many edge constructs do you use in your models? Why is that the best answer for this design?

Only one edge-sensitive construct is used in the **register_file_ram** module: always **@(posedge ws)** for writing data. This is optimal because memory writes typically occur synchronously on a clock or strobe signal, ensuring data is latched reliably. All reads are handled combinationally using always **@(*)** to avoid unnecessary timing constraints.

RESULTS:

The following key test cases were verified successfully:

RAM Module:

a) All 32 memory locations (0 to 31) were successfully written with sequential data.
b) Each memory location was read back to confirm correctness.
c) The module correctly entered high-impedance state when either cs = 1 or oe = 0.
d) "Walking Ones" pattern (00000001, 00000010, ..., 10000000) was written and read correctly.
e) Attempting to write with cs = 1 correctly resulted in no data being stored.

ROM Module:

a) ROM initialized with values from rom_data.mem.
b) Memory locations 0x04 to 0x0F were read and validated.
c) Uninitialized locations (0x00 to 0x03) correctly displayed undefined data.
d) Data from locations 0x10 to 0x17 was **scrambled** using bit manipulation and saved in a mirror model.

Waveform screenshots (attached) and log files (tb1.log, tb2.log) provide a full trace of

all operations which can be found in Appendices.

CONCLUSION

This lab successfully demonstrated the design of memory subsystems in Verilog using parameterizable registers. The RAM module effectively handled read/write logic with bi-directional communication and output control. The ROM module accurately loaded external memory content and supported read operations with proper high-impedance handling. All test scenarios were verified, including edge cases and data scrambling.

The design meets all specified criteria and shows the fundamental behaviors of RAM/ROM systems used in digital hardware design.

APPENDIX

Appendix A: register_file_ram.v

```
/*************************************************************************
*** EE 526 L Experiment #7            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #7 Design and Verification of RAM and ROM using        ***
*** registers.                                                        ***
***                                                                   ***
*** Filename: register_file_ram.v                                     ***
***                        Created by Muhammad Umar, 04/09/2025       ***
***                                                                   ***
*** This module implements a register file using RAM with separate    ***
*** control for chip select (cs), output enable (oe), and write       ***
*** strobe (ws). It supports bidirectional data transfer through      ***
*** an inout bus,allowing read and write operations based on control  ***
*** signals. The memory depth and data width are parameterizable       ***
*************************************************************************/
`timescale 1ns / 1ps

module register_file_ram #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 5,
    parameter DEPTH = 1 << ADDR_WIDTH
)(
    inout [DATA_WIDTH-1:0] data,
    input [ADDR_WIDTH-1:0] addr,
    input cs,      // active low
    input oe,      // active high
    input ws       // write strobe (write on posedge)
);

    reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];
    reg [DATA_WIDTH-1:0] read_data;

    // Write Operation (triggered on write strobe)
    always @(posedge ws) begin
        if (!cs && !oe) begin
            mem[addr] <= data;
        end
    end

    // Combinational Read Logic
    always @(*) begin
        if (!cs && oe)
            read_data = mem[addr];
```

```
        else
            read_data = {DATA_WIDTH{1'bz}};
    end

    // Drive data bus only during read
    assign data = (!cs && oe) ? read_data : {DATA_WIDTH{1'bz}};

endmodule
```

Appendix B: tb_register_file_ram.v

```
/*************************************************************************
*** EE 526 L Experiment #7            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #7 Design and Verification of RAM and ROM using        ***
*** registers.                                                        ***
***                                                                   ***
*** Filename: tb_register_file_ram.v                                  ***
***                         Created by Muhammad Umar, 04/09/2025      ***
***                                                                   ***
*** This testbench verifies the functionality of the register_file_ram***
*** module by simulating various read/write operations. It exercises  ***
*** sequential writes, individual and block reads, disabled-state     ***
*** behavior, and bus high-impedance conditions. It also includes     ***
*** pattern tests like walking ones for functional validation.        ***
*************************************************************************/

`timescale 1ns / 1ps

module tb_register_file_ram;

    // Parameters
    parameter DATA_WIDTH = 8;
    parameter ADDR_WIDTH = 5;
    parameter DEPTH = 1 << ADDR_WIDTH;

    reg [ADDR_WIDTH-1:0] addr;
    reg cs;
    reg oe;
    reg ws;
    wire [DATA_WIDTH-1:0] data;
    reg [DATA_WIDTH-1:0] data_driver;
    reg data_drive_en;
    integer i;

    assign data = data_drive_en ? data_driver : {DATA_WIDTH{1'bz}};

    register_file_ram #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) dut (
        .data(data),
        .addr(addr),
        .cs(cs),
        .oe(oe),
```

```
        .ws(ws)
    );

    initial begin
   $vcdpluson;
        $display("================================================");
        $display(" Starting RAM Testbench for register_file_ram");
        $display("================================================");

        addr = 0;
        cs = 1; oe = 0; ws = 0;
        data_driver = 0;
        data_drive_en = 0;

        #10;

        // A. Write to all memory locations

$display("\n============================================================"
);
        $display(" Writing 0 to 31 into addresses 0 to 31
(sequential)");

$display("============================================================"
);
        cs = 0; oe = 0;
        for (i = 0; i < DEPTH; i = i + 1) begin
            addr = i;
            data_driver = i;
            data_drive_en = 1;
            #5; ws = 1; #5; ws = 0; #5;
            $display("  Time %0t | WROTE %0d at Address %0d", $time,
data_driver, addr);
        end
        data_drive_en = 0;

        // A, E. Read all back (individual read)

$display("\n============================================================
======");
        $display(" Reading back values (verifying uniqueness for 32
locations)");
```

```
$display("============================================================
====");
        oe = 1;
        for (i = 0; i < DEPTH; i = i + 1) begin
            addr = i;
            #10;
            $display("  Time %0t | READ %0d from Address %0d", $time,
data, addr);
        end
        oe = 0;

        // B. Block Read (keep oe high)
        $display("\n=============================================");
        $display(" Block Read test (keep oe high)");
        $display("=============================================");
        oe = 1;
        for (i = 0; i < 5; i = i + 1) begin
            addr = i;
            #10;
            $display("  Time %0t | Output Enable(oe)=%b BLOCK READ
Address %0d => %h", $time,oe, i, data);
        end
        oe = 0;

        // C. Test Disabled State

$display("\n============================================================
====");
        $display(" Testing disabled states (should ignore oe/ws when
cs = 1)");

$display("============================================================
==");
        cs = 1; oe = 1; ws = 1;
        addr = 0;
        data_drive_en = 1;
        data_driver = 8'hAA;  // try writing this
        #10; ws = 0; #10;
        $display("  Time %0t | cs=1 writing = %0h to Address = %0d
should be ignored", $time, data_driver, addr);
        data_drive_en = 0;

        // Read back and confirm not written
```

```
        $display("\n==========================================");
        $display(" Read back and confirm not written");
        $display("==========================================");
        cs = 0; oe = 1;
        addr = 0;
        #10;
        $display("  Time %0t | Value at Address 0 (should still be
0): Data = %h", $time, data);
        oe = 0;

        // D. High-Z Test
        $display("\n==========================================");
        $display(" High-Z test When Chip Select is Active (cs=1)");
        $display("==========================================");
        addr = 0;
        cs = 1; oe = 1;
        #10;
        if (data === {DATA_WIDTH{1'bz}})
            $display("  Time %0t Chip Select(cs) = %b | PASS: Data
bus is high-Z as expected: %b", $time, cs, data);
        else
            $display("  Time %0t Chip Select(cs) = %b | FAIL: Data
bus is not high-Z: %b", $time, cs, data);
        cs = 0; oe = 0;


   // E. High-Z when OE goes low (cs = 0)

$display("\n==========================================================
=====");
        $display(" Checking that data bus becomes High-Z when oe goes
low (cs = 0)");

$display("==========================================================
===");
        addr = 0;
        cs = 0;
        oe = 1;
        #10;
        $display("  Time %0t | oe=1 Data driven = %b", $time, data);

        oe = 0; // output will go to high-Z
        #10;
```

```
        if (data === {DATA_WIDTH{1'bz}})
            $display("  Time %0t | oe=0 PASS: Data bus returned to
High-Z: %b", $time, data);
        else
            $display("  Time %0t | oe=0 FAIL: Data bus is not High-Z:
%b", $time, data);

        // F. Walking Ones Pattern (4-bit pattern to 4 addresses)
        $display("\n=============================================");
        $display(" Writing 8-bit Walking Ones pattern");
        $display("=============================================");
        for (i = 0; i < 8; i = i + 1) begin
            addr = i;
            data_driver = 1 << i;
            data_drive_en = 1;
            #5; ws = 1; #5; ws = 0; #5;
            $display("  Time %0t | WROTE %b at Address %0d", $time,
data_driver, addr);
        end
        data_drive_en = 0;

        $display("\n=============================================");
        $display(" Reading back Walking Ones pattern");
        $display("=============================================");
        oe = 1;
        for (i = 0; i < 8; i = i + 1) begin
            addr = i;
            #10;
            $display("  Time %0t | READ %b from Address %0d", $time,
data, addr);
        end
        oe = 0;

        $display("\n Test Cases Completed.");
        $display("=============================================\n");
        $finish;
    end

endmodule
```

Appendix C: register_file_rom.v

```
/*************************************************************************
*** EE 526 L Experiment #7            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #7 Design and Verification of RAM and ROM using        ***
*** registers.                                                        ***
***                                                                   ***
*** Filename: register_file_rom.v                                     ***
***                        Created by Muhammad Umar, 04/09/2025       ***
***                                                                   ***
*** This module implements a read-only memory (ROM) block with        ***
*** parameterizable data width and depth. Contents are initialized    ***
*** from an external .mem file at simulation start. The module        ***
*** supports   chip select (cs) and output enable (oe) controls, and  ***
*** outputs data  in a high-impedance state when disabled.            ***
*************************************************************************/

`timescale 1ns / 1ps

module register_file_rom #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 5,
    parameter DEPTH = 1 << ADDR_WIDTH
)(
    output [DATA_WIDTH-1:0] data,
    input [ADDR_WIDTH-1:0] addr,
    input cs,  // active low
    input oe   // active high
);

    reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];

    // Initialize ROM contents from file
    initial begin
        $readmemh("rom_data.mem", mem);
    end

    // Output logic
    assign data = (!cs && oe) ? mem[addr] : {DATA_WIDTH{1'bz}};

endmodule
```

Appendix D: tb_register_file_rom.v

```
/*************************************************************************
*** EE 526 L Experiment #7            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #7 Design and Verification of RAM and ROM using        ***
*** registers.                                                        ***
***                                                                   ***
*** Filename: tb_register_file_rom.v                                  ***
***                        Created by Muhammad Umar, 04/09/2025       ***
***                                                                   ***
*** This testbench verifies the register_file_rom module by reading   ***
*** initialized memory contents and observing outputs based on        ***
*** address. It tests both defined and undefined locations, simulates  ***
*** bit scrambling into a mirror model, and confirms data read        ***
*** behavior  under active chip select and output enable signals.      ***
*************************************************************************/

`timescale 1ns / 1ps

module tb_register_file_rom;

    parameter DATA_WIDTH = 8;
    parameter ADDR_WIDTH = 5;
    parameter DEPTH = 1 << ADDR_WIDTH;

    reg [ADDR_WIDTH-1:0] addr;
    reg cs;
    reg oe;
    wire [DATA_WIDTH-1:0] data;

    // Instantiate ROM
    register_file_rom #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) dut (
        .data(data),
        .addr(addr),
        .cs(cs),
        .oe(oe)
    );

    // Internal RAM model to simulate writing scrambled data
    reg [DATA_WIDTH-1:0] mirror_mem [0:DEPTH-1];
   integer i;
```

```
    function [7:0] scramble_bits;
        input [7:0] in_byte;
        begin
            scramble_bits = {in_byte[0], in_byte[7], in_byte[1],
in_byte[6], in_byte[2], in_byte[5], in_byte[3], in_byte[4]};
        end
    endfunction

    initial begin
        $vcdpluson;
        $display("================================================");
        $display(" Starting ROM Testbench for register_file_rom");
        $display("================================================");

        cs = 0;
        oe = 1;
        addr = 0;

        #10;

        // Demonstrating initialized data
   $display("\n================================================");
        $display(" Initialized Memory for Checking (0x04 to 0x0F)");
        $display("================================================");

        for (i = 4; i <= 15; i=i+1) begin
            addr = i;
            #5;
            $display("Address %02h = %02h", i, data);
        end

        // Demonstrate undefined locations
   $display("\n================================================");
        $display(" Unspecified Addresses Checking (should be X)");
        $display("================================================");
        for (i = 0; i < 4; i=i+1) begin
            addr = i;
            #5;
            $display("Address %02h = %02h", i, data);
        end

        // Scramble and simulate write to mirror model
   $display("\n================================================");
        $display(" Scramble and Rewrite (0x10 to 0x17)");
```

```
        $display("================================================");
        for (i = 16; i <= 23; i=i+1) begin
            addr = i;
            #5;
            mirror_mem[i] = scramble_bits(data);
            $display("Original %02h Scrambled = %02h", data,
mirror_mem[i]);
        end

        //  Show scrambled memory content
   $display("\n================================================");
        $display(" Scrambled Values");
        $display("================================================");
        for (i = 16; i <= 23; i = i+1) begin
            $display("Address %02h = %02h", i, mirror_mem[i]);
        end

        // Final memory
   $display("\n================================================");
        $display(" Full Memory ");
        $display("================================================");
        for (i = 0; i < DEPTH; i=i+1) begin
            addr = i;
            #2;
            $display("ROM[%02h] = %02h", i, data);
        end

        $display("Completed Test Cases for ROM");
        $finish;
    end

endmodule
```

Appendix E: tb1_ram.log

```
Command: /home/users15/mu832615/526LSP25/LAB7/./simv -l tb1_ram.log
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version U-2023.03-SP1_Full64; Runtime version U-2023.03-
SP1_Full64;  Apr 13 01:40 2025
VCD+ Writer U-2023.03-SP1_Full64 Copyright (c) 1991-2023 by Synopsys Inc.
====================================================
 Starting RAM Testbench for register_file_ram
====================================================

============================================================
 Writing 0 to 31 into addresses 0 to 31 (sequential)
============================================================
  Time 25000 | WROTE 0 at Address 0
  Time 40000 | WROTE 1 at Address 1
  Time 55000 | WROTE 2 at Address 2
  Time 70000 | WROTE 3 at Address 3
  Time 85000 | WROTE 4 at Address 4
  Time 100000 | WROTE 5 at Address 5
  Time 115000 | WROTE 6 at Address 6
  Time 130000 | WROTE 7 at Address 7
  Time 145000 | WROTE 8 at Address 8
  Time 160000 | WROTE 9 at Address 9
  Time 175000 | WROTE 10 at Address 10
  Time 190000 | WROTE 11 at Address 11
  Time 205000 | WROTE 12 at Address 12
  Time 220000 | WROTE 13 at Address 13
  Time 235000 | WROTE 14 at Address 14
  Time 250000 | WROTE 15 at Address 15
  Time 265000 | WROTE 16 at Address 16
  Time 280000 | WROTE 17 at Address 17
  Time 295000 | WROTE 18 at Address 18
  Time 310000 | WROTE 19 at Address 19
  Time 325000 | WROTE 20 at Address 20
  Time 340000 | WROTE 21 at Address 21
  Time 355000 | WROTE 22 at Address 22
  Time 370000 | WROTE 23 at Address 23
  Time 385000 | WROTE 24 at Address 24
  Time 400000 | WROTE 25 at Address 25
  Time 415000 | WROTE 26 at Address 26
  Time 430000 | WROTE 27 at Address 27
  Time 445000 | WROTE 28 at Address 28
  Time 460000 | WROTE 29 at Address 29
  Time 475000 | WROTE 30 at Address 30
  Time 490000 | WROTE 31 at Address 31

================================================================
 Reading back values (verifying uniqueness for 32 locations)
```

```
================================================================
  Time 500000 | READ 0 from Address 0
  Time 510000 | READ 1 from Address 1
  Time 520000 | READ 2 from Address 2
  Time 530000 | READ 3 from Address 3
  Time 540000 | READ 4 from Address 4
  Time 550000 | READ 5 from Address 5
  Time 560000 | READ 6 from Address 6
  Time 570000 | READ 7 from Address 7
  Time 580000 | READ 8 from Address 8
  Time 590000 | READ 9 from Address 9
  Time 600000 | READ 10 from Address 10
  Time 610000 | READ 11 from Address 11
  Time 620000 | READ 12 from Address 12
  Time 630000 | READ 13 from Address 13
  Time 640000 | READ 14 from Address 14
  Time 650000 | READ 15 from Address 15
  Time 660000 | READ 16 from Address 16
  Time 670000 | READ 17 from Address 17
  Time 680000 | READ 18 from Address 18
  Time 690000 | READ 19 from Address 19
  Time 700000 | READ 20 from Address 20
  Time 710000 | READ 21 from Address 21
  Time 720000 | READ 22 from Address 22
  Time 730000 | READ 23 from Address 23
  Time 740000 | READ 24 from Address 24
  Time 750000 | READ 25 from Address 25
  Time 760000 | READ 26 from Address 26
  Time 770000 | READ 27 from Address 27
  Time 780000 | READ 28 from Address 28
  Time 790000 | READ 29 from Address 29
  Time 800000 | READ 30 from Address 30
  Time 810000 | READ 31 from Address 31

============================================
 Block Read test (keep oe high)
============================================
  Time 820000 | Output Enable(oe)=1 BLOCK READ Address 0 => 00
  Time 830000 | Output Enable(oe)=1 BLOCK READ Address 1 => 01
  Time 840000 | Output Enable(oe)=1 BLOCK READ Address 2 => 02
  Time 850000 | Output Enable(oe)=1 BLOCK READ Address 3 => 03
  Time 860000 | Output Enable(oe)=1 BLOCK READ Address 4 => 04

================================================================
 Testing disabled states (should ignore oe/ws when cs = 1)
================================================================
  Time 880000 | cs=1 writing = aa to Address = 0 should be ignored

============================================
```

```
 Read back and confirm not written
================================================
  Time 890000 | Value at Address 0 (should still be 0): Data = 00

================================================
 High-Z test When Chip Select is Active (cs=1)
================================================
  Time 900000 Chip Select(cs) = 1 | PASS: Data bus is high-Z as expected:
zzzzzzzz

=====================================================================
 Checking that data bus becomes High-Z when oe goes low (cs = 0)
=====================================================================
  Time 910000 | oe=1 Data driven = 00000000
  Time 920000 | oe=0 PASS: Data bus returned to High-Z: zzzzzzzz

================================================
 Writing 8-bit Walking Ones pattern
================================================
  Time 935000 | WROTE 00000001 at Address 0
  Time 950000 | WROTE 00000010 at Address 1
  Time 965000 | WROTE 00000100 at Address 2
  Time 980000 | WROTE 00001000 at Address 3
  Time 995000 | WROTE 00010000 at Address 4
  Time 1010000 | WROTE 00100000 at Address 5
  Time 1025000 | WROTE 01000000 at Address 6
  Time 1040000 | WROTE 10000000 at Address 7

================================================
 Reading back Walking Ones pattern
================================================
  Time 1050000 | READ 00000001 from Address 0
  Time 1060000 | READ 00000010 from Address 1
  Time 1070000 | READ 00000100 from Address 2
  Time 1080000 | READ 00001000 from Address 3
  Time 1090000 | READ 00010000 from Address 4
  Time 1100000 | READ 00100000 from Address 5
  Time 1110000 | READ 01000000 from Address 6
  Time 1120000 | READ 10000000 from Address 7

 Test Cases Completed.
================================================

$finish called from file "tb_register_file_ram.v", line 163.
$finish at simulation time              1120000
           V C S   S i m u l a t i o n   R e p o r t
Time: 1120000 ps
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Sun Apr 13 01:40:33 2025
```

Appendix F: tb2_rom.log:

```
Command: /home/users15/mu832615/526LSP25/LAB7/./simv -l tb2_rom.log
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version U-2023.03-SP1_Full64; Runtime version U-2023.03-
SP1_Full64;  Apr 13 02:28 2025
VCD+ Writer U-2023.03-SP1_Full64 Copyright (c) 1991-2023 by Synopsys Inc.
====================================================
 Starting ROM Testbench for register_file_rom
====================================================

====================================================
 Initialized Memory for Checking (0x04 to 0x0F)
====================================================
Address 04 = 58
Address 05 = ed
Address 06 = b7
Address 07 = 34
Address 08 = c9
Address 09 = 8f
Address 0a = a0
Address 0b = 9b
Address 0c = 65
Address 0d = 11
Address 0e = 03
Address 0f = 4c

====================================================
 Unspecified Addresses Checking (should be X)
====================================================
Address 00 = xx
Address 01 = xx
Address 02 = xx
Address 03 = xx

====================================================
 Scramble and Rewrite (0x10 to 0x17)
====================================================
Original da Scrambled = 73
Original 7e Scrambled = 3f
Original f2 Scrambled = 75
Original 26 Scrambled = 2c
Original 86 Scrambled = 68
Original 95 Scrambled = c9
Original fd Scrambled = df
Original b1 Scrambled = c5

====================================================
 Scrambled Values
```

```
==================================================
Address 10 = 73
Address 11 = 3f
Address 12 = 75
Address 13 = 2c
Address 14 = 68
Address 15 = c9
Address 16 = df
Address 17 = c5

==================================================
 Full Memory
==================================================
ROM[00] = xx
ROM[01] = xx
ROM[02] = xx
ROM[03] = xx
ROM[04] = 58
ROM[05] = ed
ROM[06] = b7
ROM[07] = 34
ROM[08] = c9
ROM[09] = 8f
ROM[0a] = a0
ROM[0b] = 9b
ROM[0c] = 65
ROM[0d] = 11
ROM[0e] = 03
ROM[0f] = 4c
ROM[10] = da
ROM[11] = 7e
ROM[12] = f2
ROM[13] = 26
ROM[14] = 86
ROM[15] = 95
ROM[16] = fd
ROM[17] = b1
ROM[18] = xx
ROM[19] = xx
ROM[1a] = xx
ROM[1b] = xx
ROM[1c] = 12
ROM[1d] = af
ROM[1e] = 33
ROM[1f] = xx
Completed Test Cases for ROM
$finish called from file "tb_register_file_rom.v", line 99.
$finish at simulation time               194000
          V C S   S i m u l a t i o n   R e p o r t
```

```
Time: 194000 ps
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Sun Apr 13 02:28:43 2025
```

Appendix G: rom_data.mem

```
@04
58
ED
B7
34
C9
8F
A0
9B
65
11
03
4C
@10
DA
7E
F2
26
86
95
FD
B1
@1C
12
AF
33
```